

AI agents fan out into many parallel calls — can a 16 GB M5 serve them?

vLLM (MLX backend) · continuous batching · 4-bit KV cache

4 → 8

parallel requests

+28%

aggregate throughput

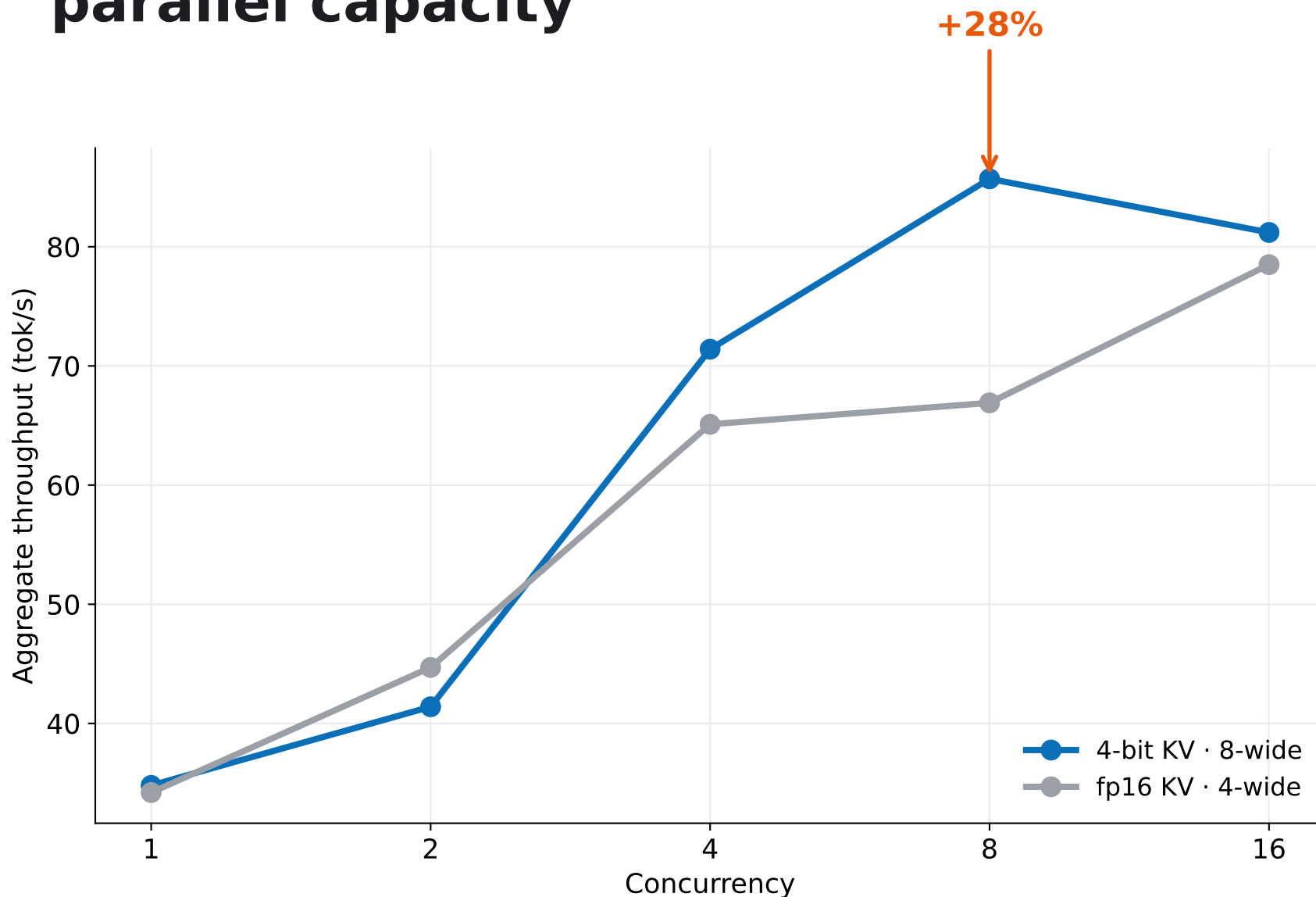
16.5s → 0.58s

first-token latency

lossless

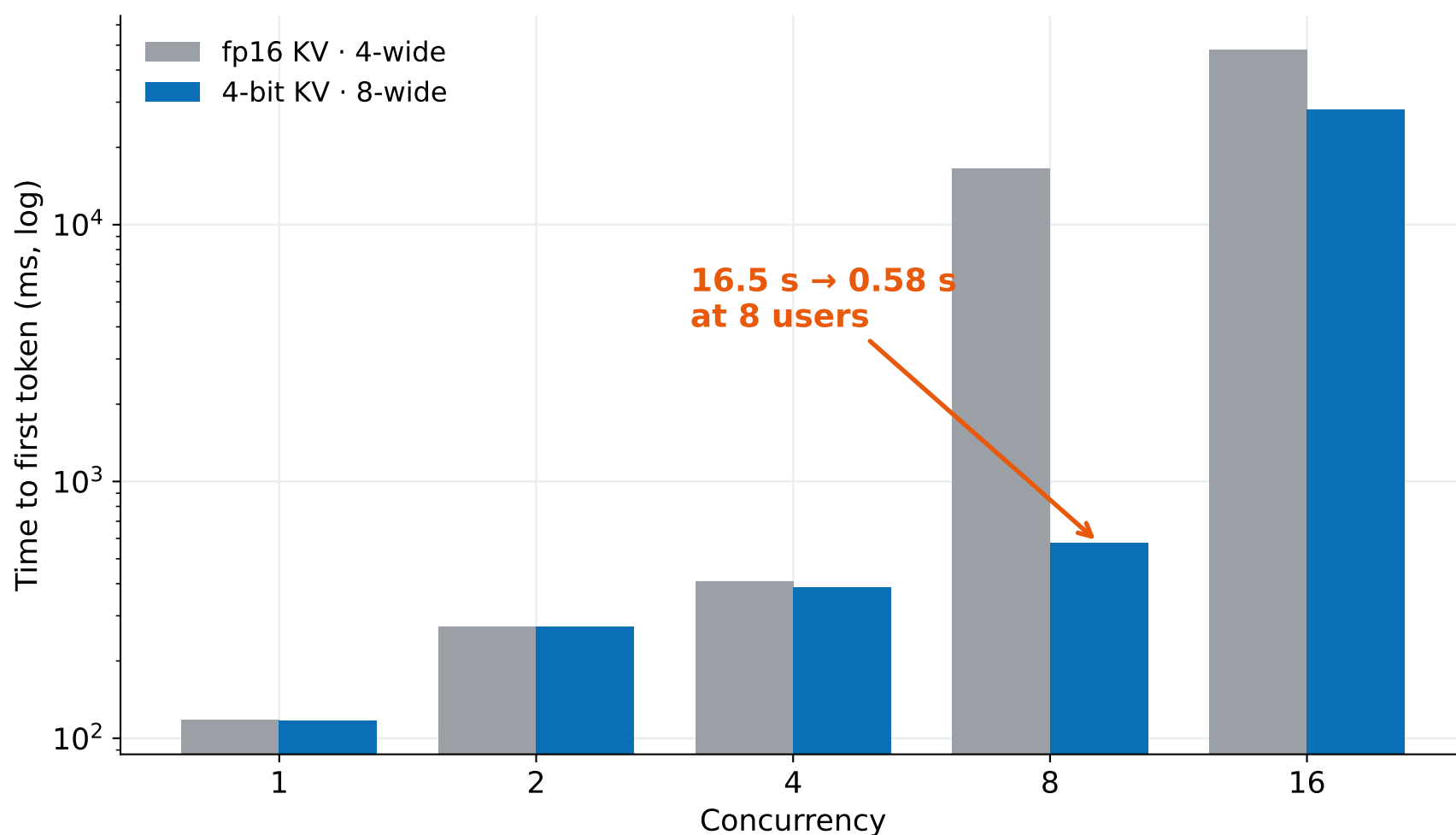
10/10 identical outputs

4-bit KV cache doubled parallel capacity



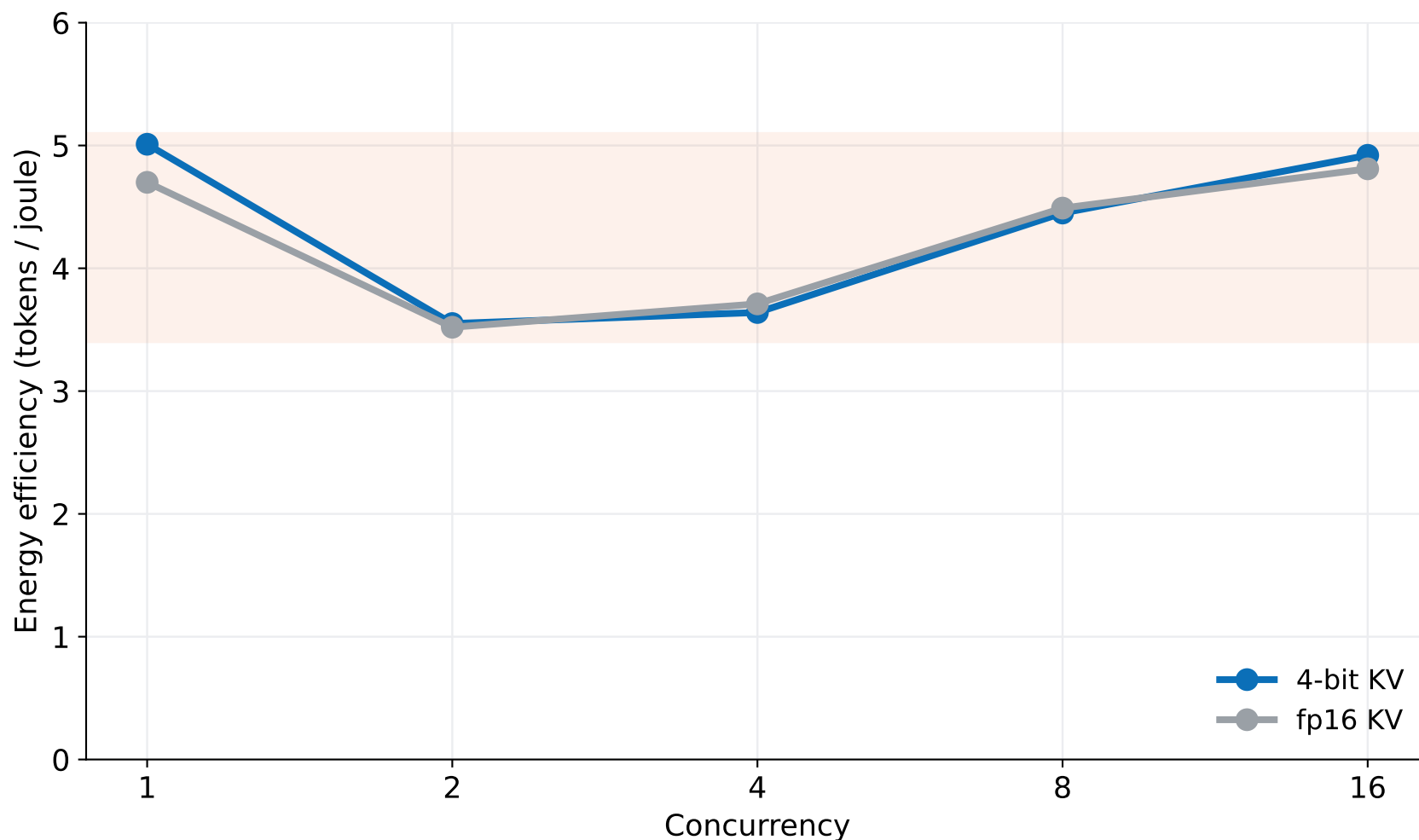
Switching the KV cache to 4-bit freed enough memory to run 8 parallel requests instead of 4 — +28% aggregate, with no single-request penalty.

A wider engine stops your agent stalling



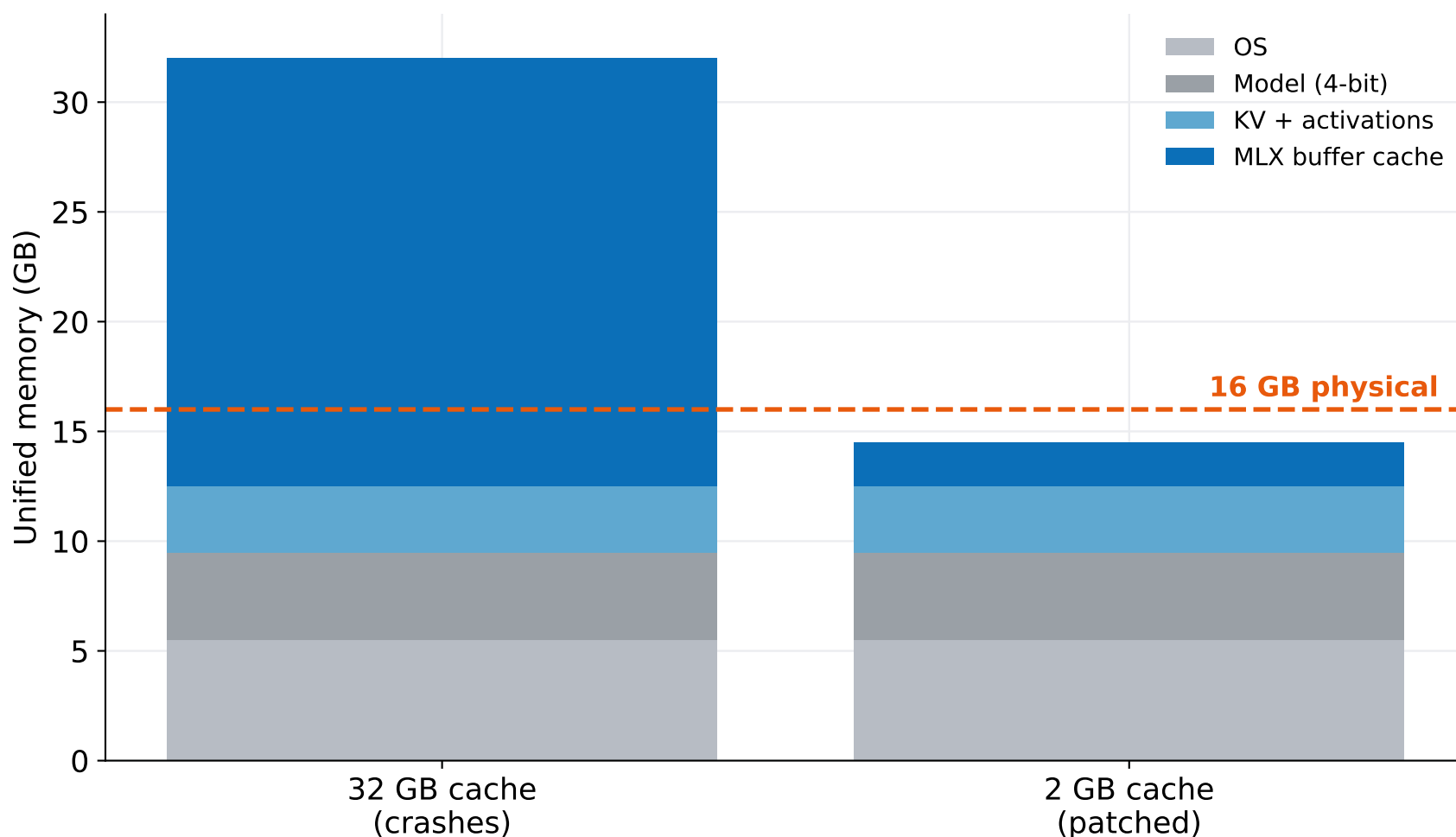
With a 4-wide engine, 4 of every 8 agent calls wait in a queue. An 8-wide engine serves them all at once — first-token latency drops 16.5 s → 0.58 s.

Throughput scales — efficiency stays flat



GPU power rises with concurrency in step with throughput, so tokens/joule stays flat (~4–5) across every load. Batch for throughput, not energy.

Bonus: a 32 GB cache limit that OOM'd 16 GB Macs



A hardcoded 32 GB MLX buffer-cache limit lets memory grow until a 16 GB Mac OOMs. Traced it, patched to 2 GB, and reported upstream. ► Full benchmark, scripts & live dashboard at github.com/rajatsadh24/vllm-mlx-qwen35-benchmark